

Array Methods - Simple Hinglish Summary

Source: <https://javascript.info/array-methods>

Main baat

Arrays ke **bahut saare methods** hain. Samajhne me aasani ke liye is chapter me unhe **groups** me baanta gaya hai.

1. Add / Remove items

Pehle se jaane hue methods:

- `arr.push(...items)`: end me jodta hai
- `arr.pop()`: end se nikalta hai
- `arr.shift()`: shuru se nikalta hai
- `arr.unshift(...items)`: shuru me jodta hai

`splice`: array ka "Swiss army knife"

Array se element hatane ke liye `delete` try karo to:

```
javascript

let arr = ["I", "go", "home"];

delete arr[1]; // "go" hata diya

alert( arr[1] ); // undefined

// ab arr = ["I", , "home"];
alert( arr.length ); // 3
```

Element hata, lekin **array ki length 3 hi rahi** (khali jagah reh gayi). Kyunki `delete obj.key` sirf key se value hata deta hai. Arrays me hum chahte hain ki baaki elements khisak kar jagah bhar dein. Isliye special methods chahiye.

`arr.splice` insert, remove aur replace, teeno kar sakta hai:

```
javascript

arr.splice(start[, deleteCount, elem1, ..., elemN])
```

Ye `arr` ko `start` index se modify karta hai: `deleteCount` elements hatata hai aur unki jagah `elem1, ..., elemN` daalta hai. **Hataye gaye elements ka array return karta hai.**

Delete:

javascript

```
let arr = ["I", "study", "JavaScript"];

arr.splice(1, 1); // index 1 se 1 element hatao

alert( arr ); // ["I", "JavaScript"]
```

Replace (3 hatao, 2 daalo):

javascript

```
let arr = ["I", "study", "JavaScript", "right", "now"];

arr.splice(0, 3, "Let's", "dance");

alert( arr ) // ["Let's", "dance", "right", "now"]
```

Return value (hataye hue elements):

javascript

```
let arr = ["I", "study", "JavaScript", "right", "now"];

let removed = arr.splice(0, 2);

alert( removed ); // "I", "study"
```

Bina hataye insert karna: `deleteCount = 0`:

javascript

```
let arr = ["I", "study", "JavaScript"];

// index 2 se, 0 delete, phir "complex" aur "language" daalo
arr.splice(2, 0, "complex", "language");

alert( arr ); // "I", "study", "complex", "language", "JavaScript"
```

Negative indexes allowed hain (end se gine jate hain):

javascript

```
let arr = [1, 2, 5];

// index -1 se, 0 delete, phir 3 aur 4 daalo
arr.splice(-1, 0, 3, 4);

alert( arr ); // 1,2,3,4,5
```

slice

`splice` se bahut simple. Ye **naya array** banata hai jisme `start` se `end` (shamil nahi) tak ke items copy hote hain:

```
javascript

arr.slice([start], [end])
```

`start` aur `end` negative bhi ho sakte hain (end se gine jate hain). String ke `str.slice` jaisa, bas substring ki jagah subarray banata hai.

```
javascript

let arr = ["t", "e", "s", "t"];

alert( arr.slice(1, 3) ); // e,s

alert( arr.slice(-2) ); // s,t
```

Bina arguments ke `arr.slice()` `arr` ki copy banata hai. Ye aksar tab kaam aata hai jab aage transformations ke liye copy chahiye jo original ko affect na kare.

concat

Naya array banata hai jisme dusre arrays ki values aur extra items hote hain:

```
javascript

arr.concat(arg1, arg2...)
```

Koi bhi number of arguments (arrays ya values). Agar argument array hai to uske **saare elements** copy hote hain, warna argument khud.

```
javascript
```

```
let arr = [1, 2];

alert( arr.concat([3, 4]) );           // 1,2,3,4
alert( arr.concat([3, 4], [5, 6]) );   // 1,2,3,4,5,6
alert( arr.concat([3, 4], 5, 6) );     // 1,2,3,4,5,6
```

Aam taur par sirf arrays ke elements copy hote hain. Array jaise dikhne wale dusre objects **poore ek item** ki tarah jode jate hain:

```
javascript

let arr = [1, 2];

let arrayLike = {
  0: "something",
  length: 1
};

alert( arr.concat(arrayLike) ); // 1,2,[object Object]
```

Lekin agar array-like object me special `Symbol.isConcatSpreadable` property ho to `concat` use array ki tarah treat karta hai:

```
javascript

let arrayLike = {
  0: "something",
  1: "else",
  [Symbol.isConcatSpreadable]: true,
  length: 2
};

alert( arr.concat(arrayLike) ); // 1,2,something,else
```

2. Iterate: `forEach`

Array ke har element ke liye function chalata hai:

```
javascript

arr.forEach(function(item, index, array) {
  // ... item ke saath kuch karo
});
```

javascript

```
// har element ke liye alert  
["Bilbo", "Gandalf", "Nazgul"].forEach(alert);
```

javascript

```
["Bilbo", "Gandalf", "Nazgul"].forEach((item, index, array) => {  
  alert(`${item} is at index ${index} in ${array}`);  
});
```

Function ka result (agar koi ho) **fenk diya jata hai, ignore** hota hai.

3. Array me search karna

`indexOf` / `lastIndexOf` aur `includes`

String ke counterparts jaise hi hain, bas characters ki jagah items par kaam karte hain:

- `arr.indexOf(item, from)`: `from` index se `item` dhundhta hai, milne par index, warna `-1`
- `arr.includes(item, from)`: milne par `true`

javascript

```
let arr = [1, 0, false];  
  
alert( arr.indexOf(0) );    // 1  
alert( arr.indexOf(false) ); // 2  
alert( arr.indexOf(null) ); // -1  
  
alert( arr.includes(1) );   // true
```

`indexOf` **strict equality** `===` use karta hai. `false` dhundho to exactly `false` milega, zero nahi. Sirf "hai ya nahi" chahiye (index nahi) to `includes` behtar hai.

`lastIndexOf`: `indexOf` jaisa, lekin **right se left** dhundhta hai:

javascript

```
let fruits = ['Apple', 'Orange', 'Apple']  
  
alert( fruits.indexOf('Apple') );    // 0 (pehla Apple)  
alert( fruits.lastIndexOf('Apple') ); // 2 (aakhri Apple)
```

`includes` `NaN` ko sahi handle karta hai, `indexOf` nahi:

javascript

```
const arr = [NaN];
alert( arr.indexOf(NaN) ); // -1 (galat)
alert( arr.includes(NaN) ); // true (sahi)
```

Kyunki `includes` bahut baad me aaya aur naya comparison algorithm use karta hai.

`find` aur `findIndex` / `findLastIndex`

Objects ke array me kisi condition se object dhundhna ho to `arr.find(fn)`:

javascript

```
let result = arr.find(function(item, index, array) {
  // true return hua to item return hota hai aur iteration ruk jata hai
  // kuch na mile to undefined
});
```

javascript

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"}
];

let user = users.find(item => item.id == 1);

alert(user.name); // John
```

Asli zindagi me objects ke arrays aam hain, isliye `find` bahut kaam ka hai. Aam taur par function ko sirf ek argument (`item`) milta hai, baaki arguments kam use hote hain.

- `findIndex`: same syntax, lekin **index** return karta hai (na mile to `-1`)
- `findLastIndex`: right se left dhundhta hai

javascript

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"},
  {id: 4, name: "John"}
];

alert(users.findIndex(user => user.name == 'John')); // 0
alert(users.findLastIndex(user => user.name == 'John')); // 3
```

filter

find ek (pehla) element dhundhta hai. Kai ho sakte hon to **arr.filter(fn)**, jo saare matching elements ka array return karta hai:

javascript

```
let results = arr.filter(function(item, index, array) {
  // true ho to item results me jata hai aur iteration chalta rehta hai
  // kuch na mile to khali array
});
```

javascript

```
let users = [
  {id: 1, name: "John"},
  {id: 2, name: "Pete"},
  {id: 3, name: "Mary"}
];

let someUsers = users.filter(item => item.id < 3);

alert(someUsers.length); // 2
```

4. Array ko transform karna

map

Sabse useful aur aksar use hone wale methods me se ek. Har element ke liye function chalata hai aur **results ka array** return karta hai:

javascript

```
let result = arr.map(function(item, index, array) {  
  // item ki jagah nayi value return karo  
});
```

javascript

```
let lengths = ["Bilbo", "Gandalf", "Nazgul"].map(item => item.length);  
alert(lengths); // 5,7,6
```

sort(fn)

Array ko **in place** sort karta hai (original badal jata hai). Sorted array return bhi karta hai, lekin aam taur par use ignore kiya jata hai.

javascript

```
let arr = [ 1, 2, 15 ];  
  
arr.sort();  
  
alert( arr ); // 1, 15, 2
```

Ajeeb natija: `[1, 15, 2]`. **By default items strings ki tarah sort hote hain.** Saare elements comparison ke liye string me convert hote hain, aur lexicographic order me `"2" > "15"`.

Apna sorting order chahiye to `arr.sort()` me function dena padta hai. Function do values compare karke ye return kare:

javascript

```
function compare(a, b) {  
  if (a > b) return 1; // pehli value badi hai  
  if (a == b) return 0; // barabar  
  if (a < b) return -1; // pehli value chhoti hai  
}
```

Numbers ke liye:

javascript

```
function compareNumeric(a, b) {  
  if (a > b) return 1;  
  if (a == b) return 0;  
  if (a < b) return -1;  
}  
  
let arr = [ 1, 2, 15 ];  
  
arr.sort(compareNumeric);  
  
alert(arr); // 1, 2, 15
```

`arr.sort(fn)` ek **generic sorting algorithm** hai (andar aksar optimized quicksort ya Timsort). Hume bas comparison karne wala `fn` dena hai.

Comparison function koi bhi number return kar sakta hai: sirf "badi" ke liye positive aur "chhoti" ke liye negative chahiye. Isliye chhota:

```
javascript  
  
arr.sort(function(a, b) { return a - b; });
```

Arrow function se sabse saaf:

```
javascript  
  
arr.sort( (a, b) => a - b );
```

Strings ke liye `localeCompare` use karo (ö jaise letters sahi sort ho):

```
javascript  
  
let countries = ['Österreich', 'Andorra', 'Vietnam'];  
  
alert( countries.sort( (a, b) => a > b ? 1 : -1 ) ); // Andorra, Vietnam, Österr  
  
alert( countries.sort( (a, b) => a.localeCompare(b) ) ); // Andorra,Österreich,
```

reverse

Array ka order ulta karta hai (in place) aur array return karta hai:

```
javascript
```

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();

alert( arr ); // 5,4,3,2,1
```

split aur **join**

Maan lo user ne comma se alag receivers ki list likhi: **John, Pete, Mary**. Humein string ki jagah names ka array chahiye.

str.split(delim) string ko delimiter se array me todta hai:

javascript

```
let names = 'Bilbo, Gandalf, Nazgul';

let arr = names.split(', ');

for (let name of arr) {
  alert( `A message to ${name}.` );
}
```

Optional dusra argument array length par limit lagata hai (kam use hota hai):

javascript

```
let arr = 'Bilbo, Gandalf, Nazgul, Saruman'.split(', ', 2);

alert(arr); // Bilbo, Gandalf
```

Khali **s** se letters me todna:

javascript

```
let str = "test";

alert( str.split('') ); // t,e,s,t
```

arr.join(glue) **split** ka ulta hai. Array ke items ko beech me **glue** lagakar ek string banata hai:

javascript

```
let arr = ['Bilbo', 'Gandalf', 'Nazgul'];

let str = arr.join(';');

alert( str ); // Bilbo;Gandalf;Nazgul
```

`reduce` / `reduceRight`

- Sirf iterate karna ho to: `forEach`, `for`, `for..of`
- Iterate karke har element ka data return karna ho to: `map`
- `reduce` aur `reduceRight` array se **ek single value** calculate karne ke liye hain.

javascript

```
let value = arr.reduce(function(accumulator, item, index, array) {
  // ...
}, [initial]);
```

Function saare elements par ek ke baad ek lagta hai aur apna **result agle call tak "le jata"** hai.

Arguments:

- `accumulator`: pichhle function call ka result. Pehli baar `initial` ke barabar (agar diya ho)
- `item`: current item
- `index`: uski position
- `array`: array

Sum ek line me:

javascript

```
let arr = [1, 2, 3, 4, 5];

let result = arr.reduce((sum, current) => sum + current, 0);

alert(result); // 15
```

Flow:

	sum	current	result
Pehla call	0	1	1
Doosra call	1	2	3
Teesra call	3	3	6
Chautha call	6	4	10
Paanchva call	10	5	15

Pichhle call ka result agle call ka pehla argument ban jata hai.

Initial value na do to bhi chalta hai (array ka pehla element initial ban jata hai aur iteration 2nd element se shuru hoti hai):

javascript

```
let result = arr.reduce((sum, current) => sum + current);
```

Lekin dhyan: array khali ho to bina initial value ke **error** aata hai:

javascript

```
let arr = [];
```

```
// Error: Reduce of empty array with no initial value
arr.reduce((sum, current) => sum + current);
```

Isliye hamesha initial value do.

`reduceRight` same kaam **right se left** karta hai.

5. `Array.isArray`

Arrays alag type nahi hain, objects par based hain. Isliye `typeof` se array aur plain object me fark nahi pata chalta:

javascript

```
alert(typeof {}); // object
alert(typeof []); // object (same)
```

Iske liye `Array.isArray(value)`:

javascript

```
alert(Array.isArray({})); // false
```

```
alert(Array.isArray([])); // true
```

6. Zyadatar methods `thisArg` support karte hain

Function call karne wale lagbhag saare methods (`find`, `filter`, `map`, ... **sirf** `sort` ko **chhodkar**) ek optional extra parameter `thisArg` lete hain.

javascript

```
arr.find(func, thisArg);  
arr.filter(func, thisArg);  
arr.map(func, thisArg);  
// thisArg aakhri optional argument hai
```

`thisArg` ki value `func` ke liye `this` ban jati hai.

javascript

```
let army = {  
  minAge: 18,  
  maxAge: 27,  
  canJoin(user) {  
    return user.age >= this.minAge && user.age < this.maxAge;  
  }  
};  
  
let users = [  
  {age: 16},  
  {age: 20},  
  {age: 23},  
  {age: 30}  
];  
  
// wo users dhundho jinke liye army.canJoin true return kare  
let soldiers = users.filter(army.canJoin, army);  
  
alert(soldiers.length); // 2  
alert(soldiers[0].age); // 20  
alert(soldiers[1].age); // 23
```

`users.filter(array.canJoin)` likhte to `array.canJoin` alag function ki tarah call hota, `this = undefined`, aur turant error aata.

`users.filter(array.canJoin, array)` ki jagah `users.filter(user => array.canJoin(user))` bhi likh sakte hain. Ye zyada use hota hai kyunki zyadatar logo ko samajhne me aasan lagta hai.

Summary: Array methods ka cheat sheet

Add/remove karne ke liye:

- `push(...items)`: end me jodta hai
- `pop()`: end se nikalta hai
- `shift()`: shuru se nikalta hai
- `unshift(...items)`: shuru me jodta hai
- `splice(pos, deleteCount, ...items)`: `pos` index par `deleteCount` elements hatata hai aur `items` daalta hai
- `slice(start, end)`: naya array banata hai, `start` se `end` (shamil nahi) tak copy karta hai
- `concat(...items)`: naya array return karta hai: current ke saare members copy karke `items` jodta hai. `items` me koi array ho to uske elements liye jate hain

Search karne ke liye:

- `indexOf/lastIndexOf(item, pos)`: `pos` se dhundhta hai, index ya `-1`
- `includes(value)`: array me `value` ho to `true`
- `find/filter(func)`: function se filter karta hai, pehli/saari values return karta hai jinke liye `true` mila
- `findIndex`: `find` jaisa, lekin value ki jagah index

Iterate karne ke liye:

- `forEach(func)`: har element ke liye `func` chalata hai, kuch return nahi karta

Transform karne ke liye:

- `map(func)`: har element par `func` ke results ka naya array
- `sort(func)`: array ko in-place sort karta hai, phir return karta hai
- `reverse()`: array ko in-place ulta karta hai, phir return karta hai
- `split/join`: string ko array me aur wapas badalta hai
- `reduce/reduceRight(func, initial)`: array par ek single value calculate karta hai, calls ke beech intermediate result pass karke

Extra:

- `Array.isArray(value)`: check karta hai ki `value` array hai ya nahi

Dhyan: `sort`, `reverse` aur `splice` array ko khud badal dete hain.

Kuch aur methods

- `arr.some(fn)` / `arr.every(fn)`: array check karte hain. `fn` har element par `map` ki tarah chalta hai. `some`: koi ek bhi result `true` ho to `true`. `every`: saare `true` hon to `true`. Ye `||` aur `&&` jaise behave karte hain (pehla result milte hi ruk jate hain). Arrays compare karne ke liye `every` use kar sakte ho:

javascript

```
function arraysEqual(arr1, arr2) {  
  return arr1.length === arr2.length && arr1.every((value, index) => value ==  
}  
  
alert( arraysEqual([1, 2], [1, 2])); // true
```

- `arr.fill(value, start, end)`: `start` se `end` index tak `value` se bharta hai
- `arr.copyWithin(target, start, end)`: `start` se `end` tak ke elements ko array ke andar hi `target` position par copy karta hai (overwrite karta hai)
- `arr.flat(depth)` / `arr.flatMap(fn)`: multidimensional array se naya flat array banate hain

Shuru me lagta hai bahut methods hain, yaad karna mushkil hai. Lekin asal me aasan hai: cheat sheet ek baar dekh lo, tasks solve karo, aur jab bhi kuch karna ho to yahan aakar sahi method dhundh lo. Jaldi hi automatically yaad ho jayenge.

Practice Tasks (Answers)

1. `border-left-width` ko `borderLeftWidth` me badlo (`camelize`):

javascript

```
function camelize(str) {  
  return str  
    .split('-')  
    .map(  
      // pehle ke ilawa baaki items ka pehla letter capital  
(word, index) => index == 0 ? word : word[0].toUpperCase() + word.slice(1  
    )  
    .join('');  
}
```

2. Range filter (`filterRange`), original nahi badalna:

javascript

```
function filterRange(arr, a, b) {  
  return arr.filter(item => (a <= item && item <= b));  
}  
  
let arr = [5, 3, 8, 1];  
  
let filtered = filterRange(arr, 1, 4);  
  
alert( filtered ); // 3,1  
alert( arr );      // 5,3,8,1 (nahi badla)
```

3. Range filter "in place" (`filterRangeInPlace`):

javascript

```
function filterRangeInPlace(arr, a, b) {  
  
  for (let i = 0; i < arr.length; i++) {  
    let val = arr[i];  
  
    // interval ke bahar ho to hata do  
    if (val < a || val > b) {  
      arr.splice(i, 1);  
      i--;  
    }  
  }  
}
```

4. Ghatte order me sort karo:

javascript

```
let arr = [5, 2, 1, -10, 8];  
  
arr.sort((a, b) => b - a);  
  
alert( arr ); // 8, 5, 2, 1, -10
```

5. Copy karke sort karo (`copySorted`):

javascript

```
function copySorted(arr) {  
  return arr.slice().sort();  
}
```

6. Extendable calculator:

javascript

```
function Calculator() {  
  
  this.methods = {  
    "-": (a, b) => a - b,  
    "+": (a, b) => a + b  
  };  
  
  this.calculate = function(str) {  
  
    let split = str.split(' '),  
        a = +split[0],  
        op = split[1],  
        b = +split[2];  
  
    if (!this.methods[op] || isNaN(a) || isNaN(b)) {  
      return NaN;  
    }  
  
    return this.methods[op](a, b);  
  };  
  
  this.addMethod = function(name, func) {  
    this.methods[name] = func;  
  };  
}
```

7. Names ka array (`map`):

javascript

```
let names = users.map(item => item.name);
```

8. Objects ko objects me map karo:

javascript

```
let usersMapped = users.map(user => ({
  fullName: `${user.name} ${user.surname}`,
  id: user.id
}));
```

Dhyan: object ke liye extra **brackets** `( )` zaruri hain, warna JS `{ }` ko function body ka shuru samjhta hai.

9. Age se users sort karo:

javascript

```
function sortByAge(arr) {
  arr.sort((a, b) => a.age - b.age);
}
```

10. Array shuffle karo:

Simple lekin galat tarika:

javascript

```
function shuffle(array) {
  array.sort(() => Math.random() - 0.5);
}
```

Ye kaam karta hai, lekin **saari permutations ki probability barabar nahi hoti.**

Sahi tarika: Fisher-Yates shuffle:

javascript

```
function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    let j = Math.floor(Math.random() * (i + 1)); // 0 se i tak random index

    // array[i] aur array[j] swap karo
    [array[i], array[j]] = [array[j], array[i]];
  }
}
```

11. Average age (`reduce`):

javascript

```
function getAverageAge(users) {  
  return users.reduce((prev, user) => prev + user.age, 0) / users.length;  
}
```

12. Unique members (`unique`):

javascript

```
function unique(arr) {  
  let result = [];  
  
  for (let str of arr) {  
    if (!result.includes(str)) {  
      result.push(str);  
    }  
  }  
  
  return result;  
}
```

Ye chhote arrays ke liye theek hai. Bade arrays me `includes` baar baar poora array chalata hai ($10000 \times 10000 = 10$ crore comparisons). Aage Map/Set se optimize karenge.

13. Array se keyed object (`groupById`, `reduce` se):

javascript

```
function groupById(array) {  
  return array.reduce((obj, value) => {  
    obj[value.id] = value;  
    return obj;  
  }, {})  
}
```

Quick Summary

Kaam	Method
End me jodo / hatao	<code>push</code> / <code>pop</code>
Shuru me jodo / hatao	<code>unshift</code> / <code>shift</code>
Beech se hatao / daalo / badlo	<code>splice(pos, deleteCount, ...items)</code>
Copy / subarray	<code>slice(start, end)</code>
Arrays jodo	<code>concat</code>
Har element par kaam	<code>forEach</code>
Hai ya nahi	<code>includes</code>
Index dhundho	<code>indexOf</code> , <code>findIndex</code>
Ek object dhundho	<code>find</code>
Sab matching dhundho	<code>filter</code>
Har element badlo	<code>map</code>
Sort	<code>sort((a, b) => a - b)</code>
Ula	<code>reverse</code>
String ↔ array	<code>split</code> / <code>join</code>
Ek value nikalo	<code>reduce((acc, item) => ..., initial)</code>
Array check	<code>Array.isArray(value)</code>
Koi ek / sab	<code>some</code> / <code>every</code>

Yaad rakho:

- `sort`, `reverse`, `splice` **original array badal dete hain.**
- `sort()` by default **strings ki tarah** sort karta hai, numbers ke liye function do.
- `reduce` me hamesha **initial value** do.
- `slice()` (bina arguments) = array ki copy.